

# SYNAP Case Study

## Case Study: Sharing Credit-Risk Data Under Regulatory Constraint

### What this document is — and isn't

The company below is fictional. Every technical result attached to it is not: it is the real, unedited output of a single, reproducible run of the 8-module pipeline ( `validation/run_validation.py` ), the same run behind `validation/Relatorio_Validacao_Fiabilidade_SYNAP.pdf` . No number in this document was adjusted, rounded favorably, or cherry-picked from a better run — anyone can regenerate every figure cited here with the commands in §7.

The scenario is deliberately narrow and realistic — one lender, one credit risk table, one external sharing constraint — rather than an abstract list of capabilities, because that is the form in which the three conditions from `PROJECT_NARRATIVE.md` (utility, privacy, practical value) actually get tested: against a specific decision someone has to make.

### 1. The scenario

**Northbridge Consumer Finance** (fictional) is a mid-size consumer lender. Its risk team wants an external model-validation vendor to independently review its credit-risk model — but sharing the real customer table (age, income, debt, credit score, and an internal risk tier) with a third party means clearing a data-protection review that, in practice, takes months and may not clear at all for this class of data.

The team has 2,200 real, already-anonymized customer records available for this purpose — enough to work with, not enough to hand to a vendor without a review cycle. Their actual question was not “*can you generate data that looks like ours*” — a vendor demo can always be made to look right — but:

Can we get data we're allowed to share, and can we **prove**, not just assert, that a model built on it says the same thing our real model would?

### 2. Setting up the Contract (Module 1)

The team split their 2,200 records honestly, the way the tool requires: **1,496 rows as the anchor** (what the system is allowed to learn from) and **704 rows held back completely** — never touched by generation, used only afterward, by Modules 7 and 8, to check the result.

```
# schema.yaml
colunas:
  idade:      { tipo: numerico,  minimo: 18, maximo: 90 }
  rendimento: { tipo: numerico,  minimo: 0 }
  divida:     { tipo: numerico,  minimo: 0 }
  score_credito: { tipo: numerico, minimo: 300, maximo: 850 }
  risco:      { tipo: categorico, categorias: [baixo, medio, alto] }
regras:
```

```
- "rendimento > 0"
```

```
synap --schema schema.yaml \  
      --anchor-csv anchor_1496.csv \  
      --target-rows 4500 \  
      --real-holdout-csv holdout_704.csv \  
      --target-column risco
```

They also declared the causal chain their own domain knowledge already told them was true — `idade → rendimento → divida`, `rendimento → score_credito`, `divida → score_credito`, `score_credito → risco` — so Module 2 wouldn't have to (re)discover it from scratch.

### 3. Generating the seed and the dataset (Modules 2-6)

Module 2 generated a 5,000-row seed. Checked against the *true* generative correlations (available here only because this is a reproducible methodology run — Northbridge's real system would only know this from a real holdout), the seed's average correlation error was **0.043**, worst case **0.095** — small, and expected for a sample of this size.

Across the five batches that produced the 4,500-row dataset:

| Signal                                           | Result                                                                              |
|--------------------------------------------------|-------------------------------------------------------------------------------------|
| Rows tested vs. rows approved                    | 4,705 candidates tested → 4,500 approved                                            |
| Rejected — Mahalanobis distance (Module 3)       | 195                                                                                 |
| Rejected — autoencoder artifact (Module 5)       | 10                                                                                  |
| Rejected — business rule violation (Module 4)    | 0                                                                                   |
| Causal drift detected (Module 4, any checkpoint) | 0 of 5                                                                              |
| Homeostasis state (Module 6) across checkpoints  | 4× "attention" (4.8–6.7% divergence), 1× "normal" — <b>never critical (&gt;10%)</b> |

No checkpoint paused generation. The "attention" state recurring in 4 of 5 checkpoints, without ever resolving to "normal", is itself informative — it shows the same three numeric columns (`divida` in particular, at 2.5–2.9% divergence contribution every single checkpoint) sitting close to, but under, the correction threshold throughout the run. §6 returns to what that would suggest doing differently.

### 4. The first gate: Module 7

Exhaustive final validation: **4,500 / 4,500 rows** conformed to the business rule (`rendimento > 0`) — zero exceptions.

The team pointed the Twin Model at `risco` (their main classification target) and got:

|     | Synthetic (TSTR) | Real (TRTR) |
|-----|------------------|-------------|
| AUC | 0.9993           | 0.9999      |

**Fidelity Index: 0.9999.** Comfortably above the 0.90 threshold. Module 7 released the dataset.

If Northbridge had stopped here — which is exactly what a tool offering only a single-metric fidelity score would leave them free to do — they would have shared this dataset with their vendor believing it was an essentially perfect stand-in for the real thing.

## 5. The second gate: Module 8 — where the story changes

Module 8 evaluated a second, harder task from the same dataset: predicting `score_credito` itself, a numeric regression rather than the 3-class classification `risco` already made easy by construction.

| Task                                    | Metric | Synthetic (TSTR) | Real (TRTR) | Utility retention |
|-----------------------------------------|--------|------------------|-------------|-------------------|
| <code>risco</code> (classification)     | AUC    | 0.999            | 1.000       | <b>99.9%</b>      |
| <code>score_credito</code> (regression) | RMSE   | 138.7            | 70.0        | <b>50.5%</b>      |

**Average retention across both tasks: 75.2% — below the 85% threshold. Module 8 did not approve utility.**

The privacy audit, run on the same dataset, came back clean on every signal:

| Check                                            | Result          | Read                                      |
|--------------------------------------------------|-----------------|-------------------------------------------|
| DCR (synthetic → real, vs. real → real baseline) | ratio 0.963     | $\geq 0.5$ is healthy; this is close to 1 |
| Exact row matches vs. the 704-row holdout        | 0               | —                                         |
| NNDR (synthetic vs. real baseline)               | 0.843 vs. 0.819 | comparable, no anomaly                    |
| Membership Inference Attack (AUC)                | 0.537           | $\approx 0.5$ = attack fails; <b>safe</b> |

**Privacy: approved, without reservation.**

## 6. What Northbridge actually did with this

A dataset that is simultaneously “privacy: clean” and “utility: below threshold, and *specifically* worse on one task than another” is not a pass/fail outcome — it’s a decision input, and Module 8 gave Northbridge enough resolution to make a narrow one instead of an all-or-nothing one:

- **Released the dataset for the classification use case** (risk-tier model validation) — the task where synthetic and real performance are, within measurement noise, identical.
- **Withheld it for credit-score regression review** — the task where a vendor’s conclusions might legitimately differ depending on which dataset they used, which is exactly the situation a synthetic dataset must not be used for.
- **Filed the four recurring “attention”-state checkpoints on `divida`** as the concrete lead for the next iteration: a larger target row count (the report’s own recommendation — see the Validation Report §9) gives the homeostasis controller more checkpoints to correct against before the run ends, which this five-batch, 4,500-row run may simply not have allowed enough time for.

None of this required a human to *read between the lines* of a single fidelity score. It required exactly the second, independent check that Module 8 exists to provide.

## 7. Reproducing every number in this document

```
python3 validation/run_validation.py # regenerates results.json + datasets, from a fixed seed
python3 validation/build_charts.py  # regenerates the figures
python3 validation/build_report.py   # regenerates the full Validation & Reliability Report
```

`results.json` is the single source of truth for every figure quoted above — none of them are hand-typed constants in this document, and none of them require trusting this document instead of checking the

source.